School of Computer Science and Engineering
Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lecture	Experiment	In Class / Take Home
3/26) Week : (23/02/26 to 1/0	1	**Minimum number of swaps** You are given two binary strings **s1** and **s2** of equal length, and the task is to find the **minimum number** of swaps required to make them identical. The only allowed operation is swapping characters between the two strings (i.e., you can swap **s1[i]** with **s2[j]**, but not within the same string). If it is impossible to make the two strings equal through such swaps, return **-1**. **Examples:** **Input:** s1 = "1100", s2 = "1111" **Output:** 1 **Explanation:** Swap the character at index 2 of s1 with the character at index 3 of s2. After this swap both strings become "1110", so only 1 swap is needed. **Input:** s1 = "00011" , s2 = "11001" **Output:** -1 **Explanation:** It is impossible to make both strings equal, so the answer is -1. Link: https://www.geeksforgeeks.org/problems/minimum-number-of-swaps--164524/1?page=1&category=Strings&sortBy=latest	In Class

	1	**Min Add to Make Parentheses Valid** You are given a string **s** consisting only of the characters **'('** and **')'**. Your task is to determine the **minimum** number of parentheses (either '(' or ')') that must be inserted at any positions to make the string s a **valid parentheses string**. A parentheses string is considered **valid** if: 1. Every opening parenthesis '(' has a corresponding closing parenthesis ')'. 2. Every closing parenthesis ')' has a corresponding opening parenthesis '('. 3. Parentheses are properly nested. **Examples:** **Input:** s = "(()(" **Output:** 2 **Explanation:** There are two unmatched '(' at the end, so we need to add two ')' to make the string valid. **Input:** s = ")))" **Output:** 3 **Explanation:** Three '(' need to be added at the start to make the string valid. **Input:** s = ")()()" **Output:** 1 **Explanation:** The very first ')' is unmatched, so we need to add one '(' at the beginning. Link: [https://www.geeksforgeeks.org/problems/min-add-to-make-parentheses valid/1?page=1&category=Strings&sortBy=latest](https://www.geeksforgeeks.org/problems/min-add-to-make-parentheses-valid/1?page=1&category=Strings&sortBy=latest)	In Class
	1	**Score of Parentheses String** Given a string **s** consisting of balanced parentheses, calculate the **score** of the string based on the following rules:	Take Home

School of Computer Science and Engineering
Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lecture	Experiment	In Class / Take Home

		• "()" has a score of 1. • "AB" has a score of A + B, where A and B are balanced parentheses strings. • "(A)" has a score of 2 × score(A), where A is a balanced parentheses string. **Note:** Test cases are generated such that the score will fit within a 32-bit integer. **Examples:** **Input:** s = "()()" **Output:** 2 **Explanation:** The string str is of the form "ab", that makes the total score = (score of a) + (score of b) = 1 + 1 = 2. **Input:** s = "(()(()))" **Output:** 6 **Explanation:** The string str is of the form "(a(b))" which makes the total score = 2 × ((score of a) + 2 × (score of b)) = 2 × (1 + 2 × (1)) = 6. **Input:** s = "((()))" **Output:** 4 **Explanation:** The string str is of the form "((a))" which makes the total score = 2 × (2 × (score of a)) = 2 × (2 × (1)) = 4. Link: https://www.geeksforgeeks.org/problems/score-of-parentheses string/1?page=1&category=Strings&sortBy=latest	
	1	**Difference Check** Given an array **arr[]** of time strings in 24-hour clock format **"HH:MM:SS"**, return the **minimum difference** in **seconds** between any two time strings in the arr[]. The clock wraps around at midnight, so the time difference between "23:59:59" and "00:00:00" is **1** second. **Examples:** **Input:** arr[] = ["12:30:15", "12:30:45"] **Output:** 30 **Explanation:** The minimum time difference is 30 seconds. **Input:** arr[] = ["00:00:01", "23:59:59", "00:00:05"] **Output:** 2 **Explanation:** The time difference is minimum between "00:00:01" and "23:59:59". Link: https://www.geeksforgeeks.org/problems/difference check/1?page=1&category=Strings&sortBy=latest	Take Home
	1	**Sort by Frequency** Given a string **s**, the task is to **arrange** the string according to the **frequency** of each character, in ascending order. If two elements have the **same** frequency, then they are sorted in **lexicographical** order. **Examples:** **Input:** s = "geeksforgeeks" **Output:** forggkksseee	Take Home

Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lecture	Experiment	In Class / Take Home
		Explanation: All the characters with minimum frequency will occur first and the one with same frequency will be arranged lexicographically. **Input:** s = "abc" **Output:** abc **Explanation:** The frequency is one for all characters hence they'll be arranged lexicographically. Link: https://www.geeksforgeeks.org/problems/sort-string-according-to-increasing frequency/1?page=1&category=Strings&sortBy=latest	
	1	**Sort an array of strings according to string lengths** You are given an array **arr[]** of strings. Your have to **sort** the array in **ascending** order based on the **lengths** of the strings. If two strings have the same length, maintain their original **relative order**. **Examples:** **Input:** arr[] = ["GeeksforGeeeks", "I", "from", "am"] **Output:** ["I", "am", "from", "GeeksforGeeks"] **Explanation:** The strings are sorted in increasing order of their lengths, starting from the shortest string "I" to the longest one "GeeksforGeeeks". **Input:** arr[] = ["You", "are", "beautiful", "looking"] **Output:** ["You", "are", "looking", "beautiful"] **Explanation:** The strings are sorted by length: "You", "are", "looking", and then "beautiful", with the shortest words appearing first and the longest last. Link: https://www.geeksforgeeks.org/problems/sort-an-array-of-strings-according-to string-lengths/1?page=1&category=Strings&sortBy=latest	Take Home

	1	**Count pairs of strings with one mismatch** Given an array **arr[]** of strings of **equal length**, where each string consists of lowercase English letters, find the **total** number of **pairs of strings** that differ in exactly **one character position**. **Note:** Two strings differ in exactly one position if they have the same length and differ in exactly one index that is, all characters except one are the same. **Examples :** **Input:** arr[] = ["abc", "abd", "bbd"] **Output:** 2 **Explanation:** The valid pairs are: 1. ("abc", "abd") → differ at index 2 2. ("abd", "bbd") → differ at index 0 **Input:** arr[] = ["def", "deg", "dmf", "xef", "dxg"] **Output:** 4 **Explanation:** The valid pairs are: 1. ("def", "deg") → differ at index 2 2. ("def", "dmf") → differ at index 1 3. ("def", "xef") → differ at index 0 4. ("deg", "dxg") → differ at index 1	Take Home

Proposed Date	Lecture	Experiment	In Class / Take Home
		Input: arr[] = ["bcde", "bced", "bdce"] **Output:** 0 **Explanation:** No pair differs at exactly one position. Link: https://www.geeksforgeeks.org/problems/count-pairs-of-strings-with-one mismatch/1?page=1&category=Strings&sortBy=latest	

3/26) Week: (23/02/26 to 1/0	2	**Winner of an election** Given a lowercase string array **arr**[]. Each element in the array represents a vote cast for a candidate. Return the name of the candidate who received the maximum number of votes and the count of votes he received. In case of a tie between two or more candidates, return the **lexicographically** smallest name. Note: Return an array of strings, the winning candidate name as the first element and the vote count as the second element (typecast the count to string). **Examples :** **Input:** arr[] = ["john", "johnny", "jackie", "johnny", "john", "jackie", "jamie", "jamie", "john", "johnny", "jamie", "johnny", "john"] **Output:** ["john", "4"] **Explanation: "**john" has 4 votes casted for him, but so does "johnny". "john" is lexicographically smaller, so we print "john" and the votes he received. **Input:** n = 3 arr[] = ["andy", "blake", "clark"] **Output:** [**"**Andy", "1"] **Explanation:** All the candidates get 1 votes each. We print "andy" as it is lexicographically smaller. Link: https://www.geeksforgeeks.org/problems/winner-of-an-election-where-votes-are represented-as-candidate-names-1587115621--162858/1?page=2&category=Strings&sortBy=latest	In Class
	2	**Check if Permutation is Substring** Given two strings **txt** and **pat** having lowercase letters, the task is to check if any permutation of **pat** is a substring of **txt**. **Examples:** **Input:** txt = "geeks", pat = "eke" **Output:** true **Explanation:** "eek" is a permutation of "eke" which exists in "geeks". **Input:** txt = "programming", pat = "rain" **Output:** false **Explanation:** No permutation of "rain" exists as a substring in "programming". Link: https://www.geeksforgeeks.org/problems/check-if-permutation-is substring/1?page=1&category=Strings&sortBy=latest	In Class

School of Computer Science and Engineering
Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lec	Experiment	In Class / Take Hom

	t u r e		e
	2	**Lexicographically Largest String After K Deletions** Given a string **s** consisting of lowercase English letters and an integer **k**, your task is to remove exactly k characters from the string. The resulting string must be the largest possible in **lexicographical order**, while maintain the relative order of the remaining characters. **Examples:** **Input:** s = "ritz", k = 2 **Output:** tz **Explaination:** By removing two characters in all possible ways, we get: "ri", "rt", "rz", "it", "iz", and "tz". Among these, "tz" is lexicographically largest string. **Input:** s = "zebra", k = 3 **Output:** zr **Explaination:** Removing "e", "b", and "a" results in "zr", which is lexicographically largest string. Link: https://www.geeksforgeeks.org/problems/lexicographically-largest-string-after deleting-k-characters/1?page=1&category=Strings&sortBy=latest	Take Hom e
	2	**Count Unique Vowel Strings** You are given a lowercase string **s**, determine the total number of distinct strings that can be formed using the following rules: • Identify all **unique** vowels (a, e, i, o, u) present in the string. • For each distinct vowel, choose **exactly one** of its occurrences from s. If a vowel appears multiple times, each occurrence represents a unique selection choice. • Generate all possible permutations of the selected vowels. Each unique arrangement counts as a distinct string. Return the total number of such distinct strings. **Examples:** **Input:** s = "aeiou" **Output:** 120 **Explanation:** Each vowel appears once, so the number of different strings can form is 5! = 120. **Input:** s = "ae" **Output:** 2 **Explanation:** Pick a and e, make all orders → "ae", "ea". **Input:** s = "aacidf" **Output:** 4 **Explanation:** Vowels in s are 'a' and 'i', Pick each 'a' once with a single 'i', make all orders → "ai", "ia", "ai", "ia". Link: https://www.geeksforgeeks.org/problems/count-unique-vowel	Take Hom e

		strings/1?page=1&category=Strings&sortBy=latest	

School of Computer Science and Engineering
Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lecture	Experiment	In Class / Take Home
	2	**Count Even Letters** You are given a string **s** consisting of lowercase english letters. Your task is to count how many **distinct** characters appear an **even** number of times in the string. **Examples:** **Input:** s = "abacaba" **Output:** 2 **Explanation:** The frequency of a is 4, b is 2 and c is 1 so there are 2 characters with even frequency. **Input:** s = "zzacccz" **Output:** 0 **Explanation:**The frequency of z is 3, a is 1 and c is 3 so there are no characters with even frequency. Link: https://www.geeksforgeeks.org/problems/count-even letters/1?page=1&category=Strings&sortBy=latest	Take Home

	2	**Shortest substring containing all vowels** You are given two strings, **s1** and **s2**, where s1 contains **distinct lowercase vowels** (a, e, i, o, u), and s2 contains **lowercase English letters**. Your task is to find the **length** of the **shortest** contiguous **substring** within s2 that contains all the **vowels** present in s1 at least **once**, in any **order**. **Note:** If there is no such substring in s2, return **-1**. **Examples:** **Input:** s1 = "ae", s2 = "acbaudeq" **Output:** 4 **Explanation:** The shortest substring of "acbaudeq" that contains both vowels 'a' and 'e' from s1 = "ae" is "aude", which has length 4. **Input:** s1 = "iou", s2 = "iuixoiu" **Output:** 3 **Explanation:** The shortest substring of "iuixoiu" that contains all vowels 'i', 'o', and 'u' from s1 = "iou" is "oiu", which has length 3. **Input:** s1 = "aeiou", s2 = "uoiee" **Output:** -1 **Explanation:** The string s2 = "uoiee" is missing the vowel 'a' from s1 = "aeiou", so no substring can contain all required vowels, and the answer is -1. Link: https://www.geeksforgeeks.org/problems/shortest-substring-containing-all vowels/1?page=1&category=Strings&sortBy=latest	Take Home
	2	**Balancing Consonants and Vowels Ratio** You are given an array of strings **arr[],** where each **arr[i]** consists of lowercase english alphabets. You need to find the number of **balanced strings** in **arr[]** which can be formed by **concatinating** one or more contiguous strings of arr[]. A balanced string contains the equal number of **vowels** and **consonants**.	Take Home

School of Computer Science and Engineering
Experiment List for Programming Ability and Logic Building - 1

Proposed Date	Lecture	Experiment	In Class / Take Home

		Examples: **Input:** arr[] = ["aeio", "aa", "bc", "ot", "cdbd"] **Output:** 4 **Explanation:** arr[0..4], arr[1..2], arr[1..3], arr[3..3] are the balanced substrings with equal consonants and vowels. **Input:** arr[] = ["ab", "be"] **Output:** 3 **Explanation:** arr[0..0], arr[0..1], arr[1..1] are the balanced substrings with equal consonants and vowels. **Input:** arr[] = ["tz", "gfg", "ae"] **Output:** 0 **Explanation:** There is no such balanced substring present in arr[] with equal consonants and vowels. Link: https://www.geeksforgeeks.org/problems/balancing-consonants-and-vowels ratio/1?page=1&category=Strings&sortBy=latest	